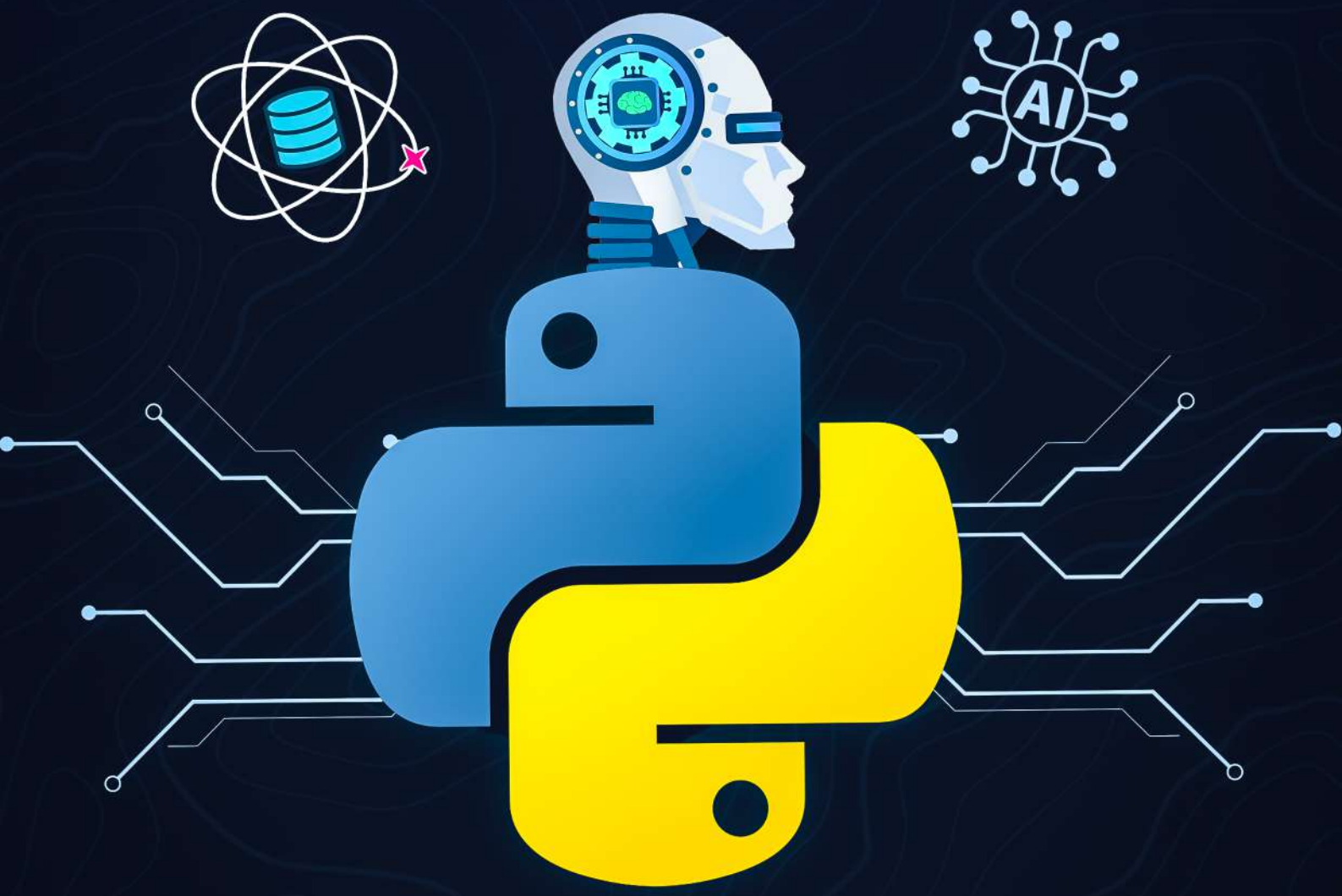
PYTHON DATA SCIENCE HANDBOOK

The Ultimate Job Ready Data Science Resource



CodeWithHarry

Introduction to Data Science

What is Data Science?

Welcome to this course on Data Science. If you are active on Fb/Insta/X, **share your Data Science journey on these platforms and tag me**. I would love to see your progress.

I am so happy you decided to learn Data Science with me in my style. Lets step back and talk a bit about why do we need data science. What kind of tasks we will do as a data scientist in an organization

What is Data Science?

At its core, **Data Science** is the field of study that uses **mathematics, statistics, programming, and domain knowledge** to extract **meaningful insights** from data. Well that sounds like a bookish definition which you are free to write in your semester exams but Data Science pretty much blends various techniques from different disciplines to analyze large amounts of information and solve real-world problems.

CodeWithHarry

Simple Definition:

Let me give you a very simple non fancy definition of Data Science

> Data Science is the process of collecting, cleaning, analyzing, and interpreting data to make informed decisions.

Why is Data Science Important?

In today's world, **data is everywhere**—from your online shopping habits to the sensors in smart devices. Companies use this data to:

- **Make better decisions** (e.g., which product to launch next).
- **Predict outcomes** (e.g., weather forecasts or stock prices).
- **Automate processes** (e.g., self-driving cars).
- **Personalize experiences** (e.g., Netflix recommendations, YouTube recommendations).

Key Steps in Data Science:

1. **Data Collection** – Gathering raw data from various sources (websites, databases, IoT devices, etc.).
2. **Data Cleaning** – Fixing errors and handling missing values (this takes up **80% of a data scientist's time**).
3. **Data Analysis** – Using statistical methods to find patterns and insights.
4. **Model Building** – Applying **machine learning** to make predictions.
5. **Interpretation & Communication** – Presenting findings in a clear way to help decision-making.

Where Do We See Data Science in Action?

- **Healthcare:** Predicting disease outbreaks and improving diagnoses.
- **E-commerce:** Personalized product recommendations.
- **Finance:** Detecting fraudulent transactions by recognizing patterns.
- **Entertainment:** Content recommendations on platforms like YouTube and Netflix.

CodeWithHarry

Why Should You Care About Data Science?

- **High Demand:** There's a global shortage of skilled data scientists.
- **Great Pay:** The average salary of a data scientist in the US is **$120,000+ per year**; in India, ₹10-30 LPA for experienced professionals.
- **Future-Proof Career:** It powers everything from AI to business strategy—this field is only growing.

Note: The content you just read will be supplied to you as a downloadable PDF. Since I am writing a summarized version of the video content which will serve as revision notes, I recommend you try to read them along with watching the videos. Thanks and see you in the next lecture!

Data Science Lifecycle

The **Data Science Lifecycle** refers to the structured process used to extract insights from data. It involves several stages, from gathering raw data to delivering actionable insights. Here is a breakdown of each step:

1. Problem Definition

> Understanding the problem you want to solve.

- Identify business objectives and define the question to answer.
- Later, we will do a very interesting project called "Coders of Delhi" where we start by understanding the business objective.
- Example: "Can we predict customer churn?" or "What factors drive sales?". In "Coders of Delhi" project, the problem is how to find potential friends of a given person in a social network

Key Activities:

- Collaborate with stakeholders.
- Define success metrics.
- Set project goals and deliverables.

CodeWithHarry

2. Data Collection

> Gathering relevant data from multiple sources.

- Sources may include databases, APIs, web scraping, or third-party datasets. Sometimes if this step is taken care of by another team or it's a data dump given by another team, we don't care where it came from.

Key Activities in data collection:

- Identify data sources (structured vs. unstructured).
- Collect data using SQL, Python, or automated pipelines.
- Ensure data relevance and completeness.

---

3. Data Cleaning (Data Preprocessing)

> Preparing raw data for analysis.

- This step addresses **missing values, duplicates, and inconsistencies**.

Key Activities:

- Handle missing or incorrect data.
- Standardize formats and remove duplicates.
- Manage outliers and inconsistencies.

Fun Fact: Data scientists spend **80% of their time** cleaning data!

---

4. Data Exploration (EDA – Exploratory Data Analysis)

> Analyzing data patterns and relationships.

- Understand data distributions and detect anomalies using visualizations.

Key Activities:

- Summarize data using statistics (mean, median, etc.).

CodeWithHarry

- Visualize patterns (using **Matplotlib**, **Seaborn**, etc.).
- Identify correlations and outliers. (correlation is how two variables move in relation to each other and outlier is a data point that stands out as unusually different from the rest. Eg. A 190 Kg heavyweight person is an outlier)

---

5. Model Building

> Creating and training machine learning models.

- Use algorithms to predict outcomes or classify data.

Key Activities:

- Choose appropriate models (e.g., regression, decision trees, neural networks).
- Split data into training and testing sets.
- Train and fine-tune models.

Common Tools: Scikit-learn, TensorFlow, PyTorch.

---

6. Model Evaluation

> Measuring model performance and accuracy.

- Evaluate models using metrics to ensure reliability.

Key Activities:

- Use performance metrics (e.g., accuracy, RMSE, ROC curve) to answer questions like - *"How often is my model correct?"*
- Perform cross-validation for robustness. Train using some part of the data and test using some part and average out the accuracy. We will study this in detail later
- Compare multiple models for best outcomes.

Key Metrics:

- Classification: Accuracy, Precision, Recall, F1-Score.

CodeWithHarry

- Regression: RMSE, R-squared.

---

7. Deployment

> Integrating the model into production systems.

- Deliver actionable results through APIs or dashboards.

Key Activities:

- Package the model for deployment (Usually done using web frameworks like Flask, and FastAPI).
- Automate pipelines for continuous learning (MLOps).
- Monitor performance post-deployment.

---

8. Communication & Reporting

> Sharing insights with stakeholders. At the end of the day the ML model solves a problem and proper reporting it to the concerned department is very important

Key Activities:

- Create dashboards
- Present findings clearly and concisely.
- Document the process and results.

---

9. Maintenance & Iteration

> Keeping the model accurate and up-to-date.

Key Activities:

- Monitor model performance.

CodeWithHarry

- Update models with new data.
- Refine features and parameters.

Summary

The **Data Science Lifecycle** is a continuous, iterative process involving:

1. Problem Definition
2. Data Collection
3. Data Cleaning
4. Data Exploration
5. Model Building
6. Model Evaluation
7. Deployment
8. Communication & Reporting
9. Maintenance & Iteration

By following this lifecycle, data scientists transform raw data into meaningful insights that drive better decision-making.

CodeWithHarry

Data Science Tools

I am enjoying teaching so far. When working in data science, the right tools make your work easier, faster, and more efficient. When I started my data science journey at IIT Kharagpur, I used to code using Pycharm and regular Python installation. I knew about Jupyter but wasn't familiar with its capabilities. From writing code to visualizing data, there are many options to choose from. Here is a breakdown of popular data science tools and why **Anaconda** with **Jupyter Notebook** is an excellent choice for beginners and advanced users.

How to run Python programs

The easiest way to run Python programs is by installing VS Code and using pip to install packages but we will use Anaconda and Jupyter notebooks

1. Jupyter Notebook (with Anaconda Distribution)

> An open-source web application that allows you to create and share documents with live code, equations, visualizations, and text.

Why Use Anaconda with Jupyter Notebook?

- **User-Friendly:** Interactive coding and easy-to-follow outputs, perfect for beginners.
- **All-in-One Package:** Anaconda includes essential libraries (NumPy, Pandas, Matplotlib + 1,500 other popular packages) pre-installed.
- **Ideal for Data Science:** Quick prototyping, data visualization, and exploratory analysis.
- **Environment Management:** Easily create isolated environments to manage package dependencies.

Common Use Cases:

- Data exploration and visualization
- Machine learning experiments
- Sharing research and reports

Installation: Althought we will do a detailed installation of Anaconda in the later sections, you can download and install the Anaconda distribution from anaconda.com. It includes Jupyter Notebook by default.

Command to Launch Jupyter Notebook:

```
jupyter notebook
```

Don't worry, we will do all these things step by step in the next section

---

2. Google Colab

> A free, cloud-based Jupyter Notebook environment provided by Google.

CodeWithHarry

Why Use Google Colab?

- **Free GPU/TPU Access:** Great for deep learning without requiring expensive hardware.
- **Cloud-Based:** No local setup—just log in and start coding.
- **Collaboration:** Share notebooks via links for easy collaboration.

Common Use Cases:

- Machine learning and deep learning projects
- Quick experiments without local setup
- Collaborative projects

Access: Use Google Colab directly in your browser at colab.research.google.com.

---

3. VS Code (Visual Studio Code)

> A lightweight and powerful code editor by Microsoft with robust extensions.

Why Use VS Code?

- **Customizable:** Extensive extensions for Python and data science (e.g., Python extension by Microsoft).
- **Integrated Jupyter Support:** You can run Jupyter Notebooks directly in VS Code.
- **Debugging Tools:** Advanced debugging capabilities.

Common Use Cases:

- Large-scale data science projects
- Working with multiple languages (Python, R, etc.)
- Integrated development (data pipelines, APIs)

Installation: Can be downloaded from code.visualstudio.com but we will install and use Anaconda distribution throughout this Data Science course.

CodeWithHarry

4. PyCharm

> A powerful, professional IDE for Python development by JetBrains.

Why Use PyCharm?

- **Professional Features:** Advanced code navigation, refactoring, and debugging.
- **Environment Management:** Virtual environment and package management built-in.
- **Scientific Mode:** Built-in support for Jupyter notebooks.

Common Use Cases:

- Large, production-level data science projects
- Building Python-based machine learning applications

Installation: Download from jetbrains.com/pycharm.

---

5. Cursor AI

> An AI-powered code editor designed for enhanced productivity with machine learning assistance.

Why Use Cursor AI?

- **AI Integration:** Code suggestions and completions for faster development.
- **Context-Aware:** Understands complex data science workflows.
- **Collaborative:** Works well with team-based projects.

Common Use Cases:

- Assisted coding for data science
- Accelerating research and prototyping
- Team collaboration

CodeWithHarry

Access: You can visit cursor.so to download cursor AI but I don't recommend using it just yet. Its important to understand the basics of data science and programming before you use such AI assistants

---

Which Tool Should You Choose?

Tool	Best For	Key Advantage
Jupyter Notebook	Interactive analysis, education	Easy to use and visualize data
Google Colab	Deep learning, cloud-based projects	Free GPU/TPU and no local setup
VS Code	Large projects, debugging	Lightweight with advanced features
PyCharm	Enterprise-level, complex applications	Professional IDE with deep features
Cursor AI	AI-assisted coding, productivity	AI-enhanced code suggestions
Spyder	Academic research, scientific computing	MATLAB-like interface

Recommendation: If you're starting out or want a hassle-free experience, **Anaconda with Jupyter Notebook** is the best choice. For advanced AI and big data projects, **Google Colab** is an excellent free alternative. For robust, large-scale development, **VS Code** or **PyCharm** provides advanced capabilities. Since we are just starting our learning journey, I will be using Anaconda and Jupyter for the most part of this course

---

Summary

1. **Anaconda + Jupyter Notebook:** Best for beginners and interactive analysis.
2. **Google Colab:** Ideal for cloud-based work and deep learning.

CodeWithHarry

3. **VS Code:** Perfect for integrated, large-scale projects.
4. **PyCharm:** A professional-grade IDE for Python development.
5. **Cursor AI:** AI-assisted productivity for fast development.

Choosing the right tool depends on your project size, complexity, and hardware needs. For most data science workflows, **Anaconda with Jupyter Notebook** offers the best balance of simplicity, flexibility, and power.

CodeWithHarry

Installing Anaconda

Anaconda is a popular Python distribution designed for data science and machine learning. It comes pre-installed with essential libraries like NumPy, pandas, and scikit-learn, along with tools like Jupyter Notebook. This makes it easy to set up a consistent environment without worrying about dependencies or compatibility issues. It's beginner-friendly and saves time by simplifying package management and project setup.

How to Install Anaconda

1. **Download Anaconda:** Go to https://www.anaconda.com/products/distribution. It might ask you for your email but there is also a "Skip Registration" button to directly download anaconda right away without email.
2. **Choose Your OS:** Select your operating system (Windows, macOS, or Linux).
3. **Download the Installer:** Click the appropriate download button (64-bit recommended).
4. **Run the Installer**
    1. **Windows/macOS**: Open the downloaded file and follow the setup instructions.
    2. **Linux**: Dowload the bash file and run it in terminal like this:

    ```
    bash Anaconda3-*.sh
    ```

    Dont forget to replace Anaconda3-* with the correct filename
5. **Verify Installation:** Open terminal or Anaconda Prompt and type:

```
conda --version
```

CodeWithHarry

You're done! It should output the version of conda you just installed

Anaconda vs Miniconda

Miniconda is a lightweight Python distribution with just conda and a few basics, ideal for custom setups. Anaconda is a full package with Python and popular data science libraries pre-installed, great for getting started quickly. Choose Miniconda for flexibility, and Anaconda for convenience.

1. Miniconda:

- **Lightweight version** of Anaconda.
- Comes with **only** `conda`, `pip`, and a few essential packages.
- When you run `conda list` after installing Miniconda, packages like **NumPy will not be present** unless you install it manually using:

  ```
  conda install numpy
  ```

2. Anaconda:

- **Full distribution** with **hundreds of packages** for data science and machine learning.
- Includes **NumPy** and many other popular libraries (like pandas, matplotlib, etc.).
- After installing Anaconda, running `conda list` will **show NumPy** by default.

Summary: Miniconda is minimal and does not come with NumPy, while Anaconda is a larger package that includes it.

CodeWithHarry

Adding conda to Path (Windows)

When you install **Anaconda** or **Miniconda**, the `conda` command-line tool is available in the terminal or shell where you ran the installation script. However, it may not work in other terminals or shells unless you add it to your **environment variables**. To **add `conda` to your environment** (so it works in any terminal or shell), follow these steps based on your operating system:

You can run this command in **Anaconda Prompt** to initialize `conda` for **Command Prompt (cmd)** or **PowerShell**:

```
conda init
```

- This modifies your shell's configuration to recognize `conda`.
- Restart your terminal after running the command.
- The process is automatic and you should be able to use conda from any directory in the terminal.

Manually (If Needed): Add these paths to the **System Environment Variables**:

1. Open:
    1. **Win + R** → Type `sysdm.cpl` to open system properties → Click **Environment Variables**.
2. Find and edit the `Path` variable in **System Variables**.
3. Add these entries:

    ```
    C:\Users\<YourUsername>\Anaconda3\Scripts
    C:\Users\<YourUsername>\Anaconda3\bin
    ```

    Dont forget to replace "<YourUsername>" with your actual username

To verify:

CodeWithHarry

```
conda --version
```

Understanding Conda Workflow

What is Conda?

Conda is a **package manager** and **environment manager** for Python (and other languages). Think of it like a **recipe manager** for your projects!

The Recipe Analogy:

Imagine you're making **mashed potatoes**. The type of **potato** you use changes the taste – in the same way, **package versions** change how your code works.

For example:

- **Python 3.11** is like using a **Shimla Aloo**.
- **Python 3.8** is like using a **Kufri Sindhuri Aloo**.

Conda lets you **control the ingredients** (package versions) for each project.

---

1. Managing Environments with Conda

By default, you are in the **base** environment.

Check Available Environments

Run the following command to list the available environments

```
conda env list
```

CodeWithHarry

2. Create a New Environment

Use this command to create a new conda environment

```
conda create -n myenv
```

Tip: You can create an environment with a specific Python version:

```
conda create -n myenv python=3.11
```

3. Activate an Environment

```
conda activate myenv
```

4. Install Packages (Example: from conda-forge Channel)

```
conda install -c conda-forge numpy
```

- `-c` flag specifies the **channel** (a source for packages).
- **conda-forge** is a popular community-maintained channel with cutting-edge packages.

5. Deactivate the Environment

```
conda deactivate
```

Summary

- **Conda** is a package and environment manager for Python.
- It lets you manage different environments for different projects.
- You can create, activate, and deactivate environments using `conda`.
- Use `conda install` to add packages to an environment.
- **Conda-forge** is a community-maintained channel with many packages.

CodeWithHarry

- **Tip:** Always activate an environment before installing packages.

Anaconda Navigator - Quick Tour

What is Anaconda Navigator

Anaconda Navigator is a graphical user interface (GUI) that comes with the Anaconda distribution. It lets you launch tools like Jupyter Notebook, Spyder, and manage environments and packages without using the command line.

To open Navigator in:

- **Windows:** Search for **Anaconda Navigator** in the Start menu.
- **macOS/Linux:** Run `anaconda-navigator` in the terminal.

It's beginner-friendly and useful for managing your data science workflow visually.

Jupyter Notebook vs. JupyterLab

Jupyter is an open-source tool for creating and sharing documents that contain **live code**, **visualizations**, **equations**, and **text**.

What is Jupyter Notebook?

- A simple, **interactive environment** to write and run code.
- Each document (called a **notebook**) is saved as a `.ipynb` file.
- Useful for **quick prototyping**, **data exploration**, and **teaching**.
- A **cell** in Jupyter Notebook is a container where you can write and run code, markdown, or raw text.

Start Jupyter Notebook by typing:

```
conda install jupyter
```

CodeWithHarry

followed by:

```
jupyter notebook
```

What is JupyterLab?

JupyterLab is the **next-generation interface** for Jupyter, offering a **more advanced** and **flexible** workspace.

Why We Use JupyterLab:

- **Multi-Panel Interface**: Open notebooks, terminals, text editors, and outputs **side by side**.
- **Better Navigation**: File browsing and workspace management are easier.
- **Extensions**: Customize with themes and advanced plugins.
- **Integrated Tools**: Combine notebooks with terminals and markdown in one window.

Start JupyterLab:

```
conda install jupyterlab
jupyter lab
```

CodeWithHarry

Key Differences:

Feature	Jupyter Notebook	JupyterLab
Interface	Single document view	Multi-tab, multi-panel
Customization	Limited	Extensive via extensions
Performance	Lightweight	Slightly heavier
Use Case	Quick tasks, tutorials	Large projects, workflows